Team Juggernauts

Jetson Bring-Up Guide

Fresh Clone → Running Localization

ROS 2 Humble · Jetson Orin Nano · FAST-LIO2 · map_localizer (VGICP)

This guide covers: what must already be installed · cloning and building on a fresh Jetson · launching the localization pipeline (and the full stack) · exactly what output reaches the controller · how to verify it is working · known open items to check before trusting results.

Companion references: **docs/testing_guide.md** (full script/launch-file/profile reference) and **docs/reuse_plan_step1.md** (the running decision log — why things are built the way they are).

1 · What Has To Already Be True Before You Start

This repo does **not** install the items below — confirm each one separately before attempting a launch.

Requirement	Why	How to Check
ROS 2 Humble installed	Everything depends on it	ls /opt/ros/humble/setup.bash
Real Hesai lidar driver (hesai_ros_driver)	challenge_master.launch.py launches it by package name — it must already be on the Jetson ROS overlay, not vendored in this repo	ros2 pkg prefix hesai_ros_driver
Real ZED SDK + zed_wrapper (Stereolabs zed-ros2-wrapper)	Same as above — camera driver is not vendored here	ros2 pkg prefix zed_wrapper
ACEINNA IMU driver running **on the RPi** (separately)	FAST-LIO2/EKF need /imu/data; this repo has no IMU package anymore — it runs independently on the RPi	Confirm /imu/data appears once bridged
Zenoh bridge running on **both** Jetson and RPi	Without it /imu/data and /wheel_odom never reach the Jetson, and /cmd_vel_nav never reaches the RPi motor driver	Lives entirely outside this repo — restart manually on both sides after any reboot
libyaml-cpp-dev, PCL, Eigen3 (system libs)	map_localizer / fast_gicp hard-require these at build time	rosdep install (part of setup.sh) pulls these in automatically

Hardware Location

On the Jetson: Hesai QT64 lidar (UDP, default 192.168.1.201:2368 — see profiles/jetson.env), ZED2i camera (USB3).

NOT on the Jetson: the IMU and the motor/CAN bus — both live on the Raspberry Pi.

2 · Clone and Build

2.1 Clone at the Recommended Path

NOTE — profiles/jetson.env's DIY_ROS_WS defaults to ${HOME}/ros2_ws, and diy_localization's map_pcd_path launch argument defaults to $DIY_ROS_WS/src/DIY-Challenge-Repo/maps/refined_map.pcd. Clone elsewhere and you must pass map_pcd_path explicitly every time you launch.

Clone with submodules, then build

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
git clone --recurse-submodules \
    https://github.com/Betsybots/DIY-Challenge-Repo.git
cd DIY-Challenge-Repo

# Builds third_party_ws (fast_gicp, ndt_omp_ros2, lidar_imu_calib,
# LIO-SAM, imu_utils_ros2_humble) + first-party DIY packages, in order.
bash setup.sh jetson
```

2.2 If setup.sh Fails Partway Through

A few known, unrelated-to-this-repo sandbox/system gaps may or may not apply to your specific Jetson image:

- **code_utils / imu_utils** failing on `elfutils/libdw.h` — missing `libdw-dev`; only affects IMU Allan-variance calibration tooling, not the main pipeline.
- **lio_sam** failing on missing **GTSAM** — only affects offline map generation (offline_mapping.launch.py), not runtime localization.

VERIFIED — Both are safe to ignore if you are not using those specific tools right now.

2.3 Sourcing Order Matters

Manual sourcing (order is important)

```
source /opt/ros/humble/setup.bash
source third_party_ws/install/setup.bash   # MUST come before the next line
source install/setup.bash
```

WARNING — diy_ndt_localization (and anything linking third_party_ws packages like ndt_omp_ros2) will fail to even be found by ros2 launch / colcon build if third_party_ws/install/setup.bash is not sourced first.

Preferred: use the env script (does this in the right order)

```
source scripts/env.sh jetson
```

3 · Launching

Option A — The Full Stack (Normal Operation)

Run on the Jetson

```
scripts/run_robot.sh jetson
```

Runs challenge_master.launch.py with every DIY_USE_* flag from profiles/jetson.env mapped to a launch argument — lidar driver, camera, FAST-LIO2, the single EKF, map_localizer, Nav2, zone_nav.

WARNING — Must be run alongside scripts/run_robot.sh raspi on the RPi (owns the motor driver, cmd_vel_mux, and the IMU) with the Zenoh bridge up on both sides. Never run the same profile on both devices — see testing_guide.md §2 / caveat #5.

Option B — Localization Only (Recommended First Test)

Isolates FAST-LIO2 + EKF + map_localizer without Nav2 / zone_nav / motor control — much easier to debug on a first run.

Run directly

```
ros2 launch diy_localization localization.launch.py \
    mode:=runtime \
    use_rviz:=true \
    map_pcd_path:=/absolute/path/to/your/refined_map.pcd
```

NOTE — map_pcd_path only needs to be passed explicitly if you did not clone at ~/ros2_ws/src/DIY-Challenge-Repo, or want to use a different map than maps/refined_map.pcd.

NOTE — This is a separate, unrelated map from the one the custom A*/PD controller stack uses. map_pcd_path here is a 3D point cloud (.pcd) for map_localizer's map→odom TF. The controller's map is a 2D occupancy grid (.pgm/.yaml) configured via map_yaml/DIY_MAP_YAML — see custom_nav_stack_design.md §5. A .pgm/.yaml pair cannot be used as a map_pcd_path value or vice versa.

What This Starts, In Order

1. **fastlio_mapping** (FAST-LIO2) — needs /lidar_points and /imu/data immediately.
2. **ekf_filter_node_odom** (the single EKF) — needs /wheel_odom (from the RPi, over Zenoh), /imu/data, and FAST-LIO2's gated output.
3. **map_localizer_node** — starts immediately but does nothing until step 4.
4. **trigger_map_relocalize.py** — waits for map_localizer_node's /relocalize service, calls it once with map_pcd_path + the initial pose (defaults to map origin), then **exits** (by design — not a long-running node).

VERIFIED — Watch for the log line: /relocalize succeeded: relocalize success — this confirms the map loaded. If instead you see it hang on "Waiting up to 30s for /relocalize service", map_localizer_node isn't up yet or crashed — check its own log for the real error.

4 · What Actually Reaches the Controller

Regardless of whether the controller is Nav2 or a custom one — there is **no controller-specific wiring** on the localization side. Two things are published, and any controller consumes them the standard ROS 2 way.

1. TF Tree: map → odom → base_link

- **map** → **odom**: broadcast by map_localizer_node (only after step 4 in §3 succeeds — before that, this transform does not exist at all).
- **odom** → **base_link**: broadcast by the EKF (ekf_filter_node_odom).

CRITICAL — There is NO /pose or /ndt_pose-style topic for global position. map_localizer only publishes TF + a debug /map_cloud + its two services. A controller that needs the robot's pose in the map frame MUST do a standard TF lookup.

C++ (rclcpp) — same pattern Nav2 itself uses internally

```
geometry_msgs::msg::TransformStamped map_to_base =
    tf_buffer_->lookupTransform("map", "base_link", tf2::TimePointZero);
```

Python (rclpy)

```
from tf2_ros import Buffer, TransformListener
tf_buffer = Buffer()
TransformListener(tf_buffer, node)
map_to_base = tf_buffer.lookup_transform(
    "map", "base_link", rclpy.time.Time())
```

2. /odometry/filtered (nav_msgs/Odometry, odom frame, 50 Hz)

Published by the EKF. Most controllers subscribe to this directly for local velocity/pose feedback (smooth, continuous, no TF-tree traversal needed) rather than doing a TF lookup on every control-loop tick.

Which One Should the Controller Use?

Need	Use This
Local smooth velocity feedback (closed-loop control)	/odometry/filtered topic
Global position for waypoint / path following against the map	map → base_link TF lookup

5 · Verifying It Is Actually Working

Topic rates you would expect

```
ros2 topic hz /lidar_points          # real Hesai QT64 lidar topic
ros2 topic hz /imu/data              # from the RPi, over Zenoh
ros2 topic hz /lidar_odometry        # FAST-LIO2 output
ros2 topic hz /odometry/filtered     # EKF output, should be ~50 Hz
```

Confirm the map actually loaded

```
ros2 service call /relocalize_check slam_interfaces/srv/IsValid \
    "{code: 1}"
```

Confirm the full TF chain resolves

```
ros2 run tf2_ros tf2_echo map base_link
```

Full pre-flight checklist (nodes, rates, TF, e-stop, Nav2)

```
scripts/health_check.sh jetson
```

WARNING — If map → base_link never resolves: check ros2 node list for map_localizer_node and ekf_filter_node_odom both present, then check ros2 topic list for /lidar_odometry and /cloud_registered_body actually publishing (both required before map_localizer's synced callback ever fires — see §3).

6 · Known Open Items — Check Before Trusting Results

These affect the localization pipeline specifically and are not yet resolved as of this guide — see reuse_plan_step1.md for full detail on each.

NOTE — RESOLVED (2026-09-11): hesai_ros_driver's real output topic is confirmed on hardware to be `/lidar_points`. All code/config in this repo now uses that name consistently (challenge_master.launch.py previously assumed /hesai/points — that assumption has been corrected; see reuse_plan_step1.md Step 25).

WARNING — extrinsic_R was just fixed to a valid rotation (identity) but not yet re-verified on hardware — the new value does not match this repo's own from-scratch Rx(-90°) derivation from an earlier accelerometer test. Re-verify with a live accelerometer reading if results look physically wrong (e.g. yaw/roll swapped).

WARNING — Whether maps/refined_map.pcd is genuinely this course's map (vs. a bench/test map) is unconfirmed. If localization looks completely wrong from the start, this is the first thing to check.

WARNING — VGICP alignment accuracy has only been verified with synthetic data in a sandbox — never against real lidar scans. Watch map_localizer's own log output and /map_cloud in RViz for obviously bad alignment.

WARNING — base_link→camera_link has NO publisher at all right now. The URDF (robot.urdf.xacro) does not define camera_link (only lidar_link/imu_link are real — verified by actually running xacro and checking the generated output), and zed_wrapper's own publish_tf was deliberately disabled based on the false assumption that the URDF already covers it. Anything projecting ZED2i image/depth data into the robot frame via TF will fail until this is fixed.

NOTE — This robot has no GPS hardware at all (confirmed) — DIY_USE_GPS is false on every profile; no /gps/fix topic will ever appear, by design, not because anything is broken.

7 · Quick Reference — Commands Used Most Often

One-time setup

```
git clone --recurse-submodules <repo-url> \
    ~/ros2_ws/src/DIY-Challenge-Repo
cd ~/ros2_ws/src/DIY-Challenge-Repo
bash setup.sh jetson
```

Every new terminal

```
source scripts/env.sh jetson
```

Test localization in isolation

```
ros2 launch diy_localization localization.launch.py \
    mode:=runtime use_rviz:=true
```

Full stack (after localization checks out)

```
scripts/run_robot.sh jetson
```

Pre-flight check

```
scripts/health_check.sh jetson
```

This guide is a focused walkthrough. For the full script / launch-file / profile reference, see docs/testing_guide.md. For why things are built the way they are, see docs/reuse_plan_step1.md.